



Table of Contents

1.Specifikáció.....	1
1.1. A feladat szöveges leírása	Hiba! A könyvjelző nem létezik.
1.2. Követelmények részletezése	Hiba! A könyvjelző nem létezik.
2. Tervezés.....	Hiba! A könyvjelző nem létezik.
2.1. Architektúra és technológiai stack	2
2.2. Konténerizációs és orkesztrációs terv	2
2.3. Skálázási és erőforrás-kezelési terv (HPA).....	3
3. Megvalósítás.....	3
3.1. A .NET Minimal API backend és statikus frontend megvalósítása	3
3.2. Multi-stage Dockerfile elkészítése	3
3.3. Kubernetes manifestek (Deployment, Service, HPA) konfigurálása	4
4. Tesztelés	5
4.1. Helyi konténerizált futtatás (Docker Desktop)	5
4.2. Kubernetes orkesztráció és HPA terhelésteszt (Minikube)	6
Összegzés.....	Hiba! A könyvjelző nem létezik.

1. Specification

1.1. Description of the Task

The objective of this project is to implement and evaluate a containerized microservice architecture within a Kubernetes environment. Using a built-in CPU stress module, the application demonstrates resource-utilization-based Horizontal Pod Autoscaling (HPA).

The system consists of two main components: a lightweight ASP.NET Core 8 Minimal API backend serving a dynamic HTML/JS dashboard, and the Kubernetes orchestration layer responsible for automated pod scaling and load balancing.



1.2. Detailed Requirements

Server-side requirements: .NET 8 runtime, provision of REST API endpoints to retrieve container metadata and to generate CPU load with configurable duration.

Client-side requirements: Pure HTML/JavaScript-based browser-accessible dashboard that interactively displays the responding container ID and triggers the load test upon button click.

Containerization requirements: Use of a multi-stage Dockerfile to reduce image size and minimize security risks.

Orchestration requirements: Kubernetes Deployment and Metrics-Server-based HPA configuration for dynamic pod scaling based on CPU utilization (min. 1, max. 5 pods).

2. Design

2.1. Architecture and Tech Stack

- Backend: C# / ASP.NET Core 8 Minimal API.
- Frontend: Static HTML5 / JavaScript (communicates with the REST API using the Fetch API).
- Containerization: Docker (Docker Desktop Engine).
- Orchestration: Kubernetes (Minikube local development cluster).

2.2. Containerization and Orchestration Plan

Multi-stage Docker build design:

- Build stage:

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
```

- Runtime stage:

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
```

Kubernetes orchestration architecture design:



Deployment: Defines the desired state of pods, the default replica count, and resource limits essential for metrics collection.

Service: Provides internal and external network access, as well as automatic load balancing across running pod instances.

2.3. Scaling and Resource Management Plan (HPA)

Kubernetes HPA makes scaling decisions based on CPU metrics collected by metrics-server.

- Pod CPU limits: Specific CPU requests and limits are configured for the pods (e.g., requests.cpu: 100m, limits.cpu: 500m).
- Autoscaling threshold: The goal of HPA is to keep the average CPU utilization of pods below 50%. Once load exceeds this threshold, new pod instances are launched.

3. Implementation

3.1. Implementation of .NET Minimal API Backend and Static Frontend

In the backend, the ordering of UseDefaultFiles() and UseStaticFiles() middleware components was configured precisely to serve static root files. Key endpoints include:

- GET /api/info: Returns the unique container identifier (Environment.MachineName) and server time.
- POST /api/stress?durationSeconds={sec}: Executes a CPU-intensive mathematical loop for the specified duration.

3.2. Creating the Multi-stage Dockerfile

The structure of the Dockerfile ensures a minimal image size:

```
1 FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
2 WORKDIR /src
3
4 COPY *.csproj ./
5 RUN dotnet restore
6
7 COPY . ./
8 RUN dotnet publish -c Release -o /app/out /p:UseAppHost=false
9
10 FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
11 WORKDIR /app
12 COPY --from=build /app/out .
13
14 EXPOSE 80
15 ENV ASPNETCORE_URLS=http://+:80
16
17 ENTRYPOINT ["dotnet", "CloudProjectApplication.dll"]
```



3.3. Kubernetes Manifest config

The manifest files located in the k8s/ directory describe the Kubernetes objects:

- Deployment: Defines the application's pods and resource limits."

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: cloud-backend-deployment
5    labels:
6      app: cloud-backend
7  spec:
8    replicas: 3
9    selector:
10     matchLabels:
11       app: cloud-backend
12   template:
13     metadata:
14       labels:
15         app: cloud-backend
16     spec:
17       containers:
18       - name: cloud-backend-container
19         image: cloud-backend:v1
20         imagePullPolicy: Never
21         ports:
22         - containerPort: 80
23         resources:
24           requests:
25             cpu: "100m"
26             memory: "64Mi"
27           limits:
28             cpu: "500m"
29             memory: "128Mi"
```

- Service: NodePort (or ClusterIP) type service for load balancing among pods.

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: cloud-backend-service
5  spec:
6    type: NodePort
7    selector:
8      app: cloud-backend
9    ports:
10     - protocol: TCP
11       port: 80
12       targetPort: 80
13       nodePort: 30080
```



- HPA: HorizontalPodAutoscaler manifest that monitors the CPU threshold.

```
1  apiVersion: autoscaling/v2
2  kind: HorizontalPodAutoscaler
3  metadata:
4    name: cloud-backend-hpa
5  spec:
6    scaleTargetRef:
7      apiVersion: apps/v1
8      kind: Deployment
9      name: cloud-backend-deployment
10   minReplicas: 1
11   maxReplicas: 5
12   metrics:
13   - type: Resource
14     resource:
15       name: cpu
16       target:
17         type: Utilization
18         averageUtilization: 50
```

4. Testing

4.1. Local Containerized Execution

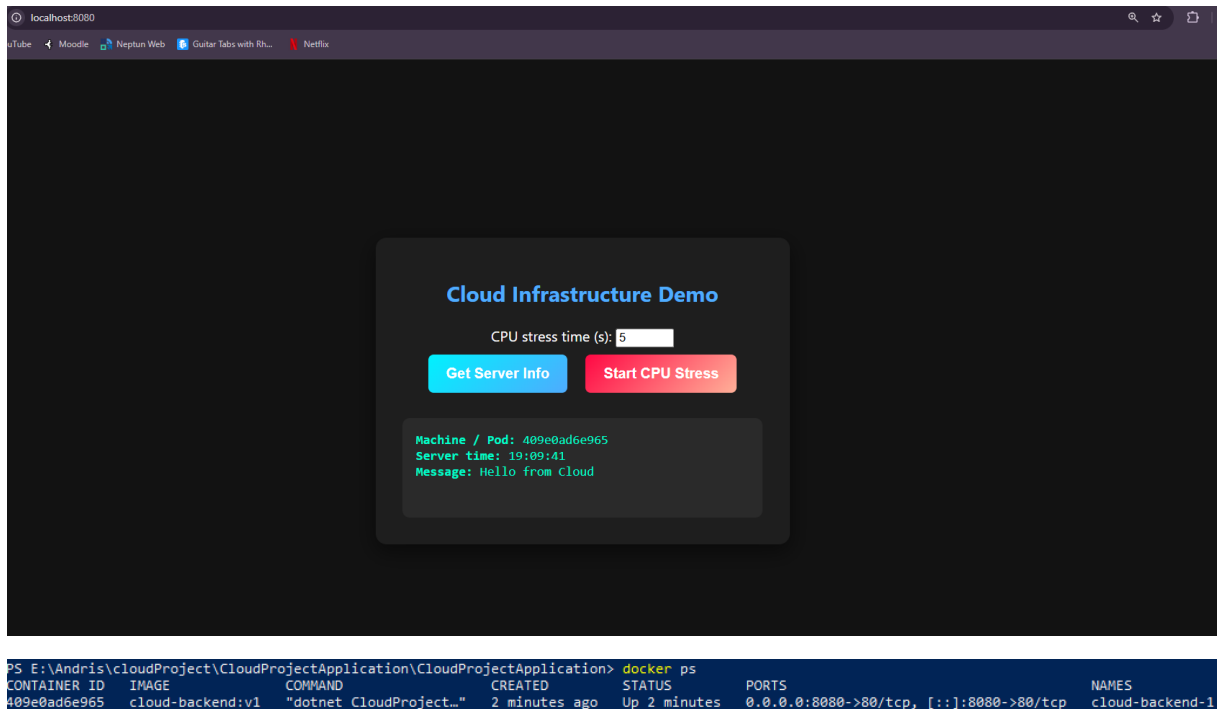
Image build:

```
docker build -t cloud-backend:v1 .
```

Create the container:

```
docker run -d -p 8080:80 --name cloud-backend-1 cloud-backend:v1
```

Checking:



The screenshot shows a web browser window at localhost:8080 displaying a 'Cloud Infrastructure Demo' application. The application has a dark theme and a central panel with the title 'Cloud Infrastructure Demo'. Below the title, there is a text input field for 'CPU stress time (s):' with the value '5'. There are two buttons: 'Get Server Info' (blue) and 'Start CPU Stress' (red). Below these buttons, a text box displays the following information: 'Machine / Pod: 409e0ad6e965', 'Server time: 19:09:41', and 'Message: Hello from Cloud'. Below the browser window, a terminal window shows the command 'docker ps' and its output, which includes the container ID '409e0ad6e965' and the image 'cloud-backend:v1'.

```
PS E:\Andris\cloudProject\CloudProjectApplication\CloudProjectApplication> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
409e0ad6e965   cloud-backend:v1 "dotnet CloudProject..." 2 minutes ago  Up 2 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  cloud-backend-1
```

The container with the correct ID is indeed running on port 8080.

4.2. Kubernetes orchestration and HPA load testing (Minikube)

Starting the Minikube cluster and loading the image:

```
minikube start --driver=docker
```

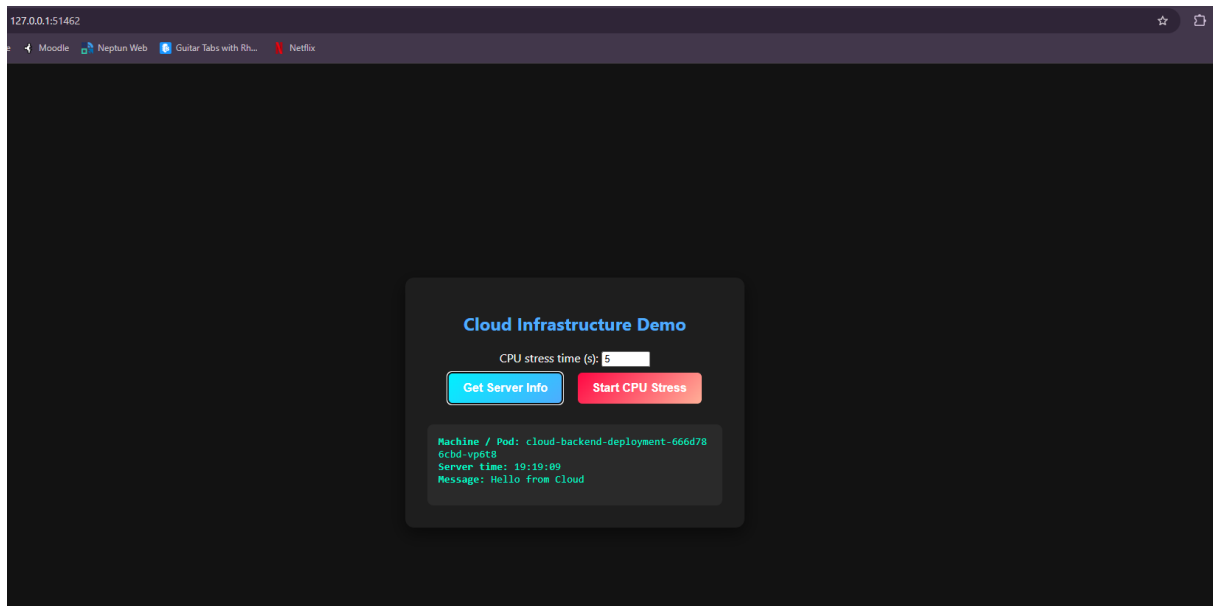
```
minikube image load cloud-backend:v1
```

Applying manifest and starting service:

```
kubectl apply -f k8s/
```

```
minikube service cloud-backend-service
```

Checking:



It can be seen that the application is already running from the pod created earlier.

Ezután a stressz teszthez...

Enable Metrics-Server:

```
minikube addons enable metrics-server
```

Observing Automatic scaling (HPA):

```
kubectl get hpa cloud-backend-hpa -w
```

Checking:

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
cloud-backend-hpa	Deployment/cloud-backend-deployment	cpu: 1%/50%	1	5	1	6m36s
cloud-backend-hpa	Deployment/cloud-backend-deployment	cpu: 112%/50%	1	5	1	8m15s
cloud-backend-hpa	Deployment/cloud-backend-deployment	cpu: 112%/50%	1	5	3	8m30s
cloud-backend-hpa	Deployment/cloud-backend-deployment	cpu: 1%/50%	1	5	3	9m15s
cloud-backend-hpa	Deployment/cloud-backend-deployment	cpu: 1%/50%	1	5	3	14m
cloud-backend-hpa	Deployment/cloud-backend-deployment	cpu: 1%/50%	1	5	1	14m

Conclusion

The tests confirmed that the Kubernetes infrastructure successfully detected the CPU load via Metrics-Server and dynamically instantiated new pods according to the HPA settings, then automatically scaled the system back down once the load subsided.

Co-author: Gemini 3.6 Flash

Lectored by: Deák András Botond